Taller Docker: contenerización, seguridad de runtime y estrés de cgroups

Programación Orientada a Objetos

Sexto Semestre

ID de estudiante: 905523

Universidad Cooperativa de Colombia

Daniel Arteaga Fajardo

21 de octubre de 2026

Resumen

Este informe documenta el diseño, la construcción y la verificación local de una imagen Docker para EcoAir Backend, una aplicación Spring Boot 3.3.5. Se implementó una construcción multietapa con Maven 3.9.6 y Java 17 en la etapa de compilación, y una imagen runtime basada en JRE 17 Alpine que incorpora únicamente el artefacto JAR. La ejecución se configuró con el usuario no privilegiado `sensor_user` (UID 10001) y con el identificador 905523 inyectado como `OPERATOR_ID`. Las pruebas de runtime confirmaron la denegación de escritura en `/etc`, la ausencia de Maven y de fuentes Java dentro de la imagen final, y una terminación por falta de memoria con `OOMKilled=true` al limitar el contenedor a 48 MiB. En la calibración, 112 MiB no satisfizo el criterio de inicio inferior a 15 segundos; 128 MiB fue el menor límite probado que sí lo cumplió y mantuvo el servicio disponible. La captura final de `docker stats` registró 122 MiB de 128 MiB (95.33 %). El punto de acceso de salud respondió con estado UP. Los resultados se respaldan en la transcripción de auditoría capturada en Docker Desktop local.

Palabras clave: Docker, Spring Boot, construcción multietapa, contenedor no root, cgroups, OOMKilled

Taller Docker: contenerización, seguridad de runtime y estrés de cgroups

La contenerización permite distribuir una aplicación con su entorno de ejecución de forma reproducible, pero la imagen resultante también debe limitar la superficie de ataque y el consumo de recursos. El taller plantea esos objetivos mediante tres verificaciones: separar construcción y runtime, comprobar que el proceso no requiere privilegios de root y observar el efecto de los límites cgroup de memoria sobre Spring Boot. En una construcción multietapa, las herramientas de compilación se mantienen en una etapa distinta y solo se copia el artefacto necesario a la etapa final (Docker Inc., s. f.-a).

El trabajo se ejecutó con Docker Desktop sobre su motor Linux/WSL2. Los resultados cuantitativos que siguen se basan en comandos reales registrados con marcas de tiempo en `audit_session.log`. Los valores de memoria y CPU de `docker stats` son observaciones puntuales del daemon y pueden variar con el tiempo; por ello se informa la hora de la captura junto con la salida.

Objetivos

Objetivo general

Construir y auditar una imagen Docker ligera, con privilegios y memoria de runtime controlados, para EcoAir Backend, verificando su funcionamiento mediante evidencia local reproducible.

Objetivos específicos

- Separar la descarga de dependencias, la compilación Maven y la ejecución del JAR en etapas de construcción y runtime.
- Ejecutar la aplicación con usuario no root y confirmar las restricciones de escritura del sistema de archivos.

- Verificar que Maven y las fuentes Java no queden dentro de la imagen de runtime.
- Observar la terminación por OOM a 48 MiB y calibrar límites de memoria hasta cumplir el criterio de inicio en menos de 15 segundos.
- Registrar identidad de imagen, variable de entorno, estado del servicio y consumo de recursos en archivos de auditoría y entrega.

Contexto y método de verificación

El backend emplea Java, Spring Boot, Spring Data JPA, una base H2 en memoria y Spring Boot Actuator. La aplicación expone el puerto 8080 y el punto de acceso de salud utilizado en la comprobación fue `/actuator/health`. La imagen fue construida con `STUDENT_ID=905523` y etiquetada como `eco-air-backend:1.0.0`.

La verificación se desarrolló en secuencia: inspección de Docker Desktop; construcción de la imagen; revisión de historial y propiedades del usuario; prueba de permisos y artefactos de desarrollo; ejecución limitada a 48 MiB; y una serie de ejecuciones con límites crecientes y decrecientes. En cada prueba de calibración se recreó el contenedor de ensayo. Al finalizar, se recreó `eco_air_active` desde la imagen final, con el límite elegido y el puerto publicado.

Diseño de la imagen y decisiones de seguridad

Construcción multietapa y caché

La etapa `builder` parte de `maven:3.9.6-eclipse-temurin-17-alpine`. Primero copia `pom.xml` y ejecuta `mvn dependency:go-offline -B`; luego copia `src/` y ejecuta `mvn -B clean package -DskipTests`. Esta secuencia permite que los cambios de código fuente no invaliden innecesariamente la capa de dependencias, siempre que `pom.xml` permanezca sin cambios.

La etapa `runtime` parte de `eclipse-temurin:17-jre-alpine`. Docker copia desde `builder` solamente `/build/target/eco-air-backend-1.0.0.jar` a `/app/app.jar`. De esta manera, Maven y las fuentes Java utilizadas durante la compilación no se copian a la imagen final, de acuerdo con el principio de seleccionar qué artefactos pasan entre etapas (Docker Inc., s. f.-a).

Usuario no privilegiado y configuración de la JVM

El Dockerfile crea el grupo `sensorgroup` y el usuario de sistema `sensor_user` con UID 10001, asigna la propiedad de `/app` y establece `USER sensor_user` antes del `ENTRYPOINT`. La comprobación con `id` dentro del contenedor confirmó `uid=10001(sensor_user)` y `gid=101(sensorgroup)`. La variable `JAVA_TOOL_OPTIONS` define `-XX:+UseContainerSupport` y `-XX:MaxRAMPercentage=75.0` para que la JVM tenga en cuenta el entorno limitado. El identificador de construcción se expone mediante `ENV OPERATOR_ID=${STUDENT_ID}`. Docker aplica los límites de memoria del contenedor a través de sus opciones de ejecución; estos límites son un control de recursos, no una garantía de que la aplicación funcione bajo cualquier límite (Docker Inc., s. f.-b).

Dockerfile aplicado

La Figura 1 presenta el Dockerfile utilizado en la construcción de la imagen.

Figura 1

Dockerfile multietapa de EcoAir Backend

```
FROM maven:3.9.6-eclipse-temurin-17-alpine AS builder
ARG STUDENT_ID
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src/ src/
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine AS runtime
ARG STUDENT_ID
RUN addgroup -S sensorgroup && adduser -S -u 10001 -G sensorgroup
sensor_user
WORKDIR /app
COPY --from=builder /build/target/eco-air-backend-1.0.0.jar /app/app.jar
RUN chown -R sensor_user:sensorgroup /app
ENV JAVA_TOOL_OPTIONS="-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0
-XX:+ExitOnOutOfMemoryError"
ENV OPERATOR_ID=${STUDENT_ID}
EXPOSE 8080
USER sensor_user
ENTRYPOINT ["/bin/sh", "-c", "exec java $JAVA_TOOL_OPTIONS -jar
/app/app.jar"]
```

Nota. El Dockerfile y el archivo `.dockerignore` fueron verificados con saltos de línea LF y sin BOM. La imagen final contiene el entorno JRE; la compilación se realiza en la etapa `builder`.

Resultados

Construcción e identidad de imagen

La imagen se construyó con el siguiente comando:

```
docker build --build-arg STUDENT_ID="905523" -t eco-air-backend:1.0.0 .
```

La construcción final terminó con código de salida 0. Una primera tentativa había fallado mientras Docker Desktop se estaba deteniendo; BuildKit devolvió un error HTTP 500 al obtener su imagen auxiliar. Tras recuperar el daemon, la compilación se completó y posteriormente se repitió después de normalizar los archivos Docker a LF. La Tabla 1 resume los datos de identidad verificados y la Figura 2 muestra las capas de la imagen en Docker Desktop.

Tabla 1

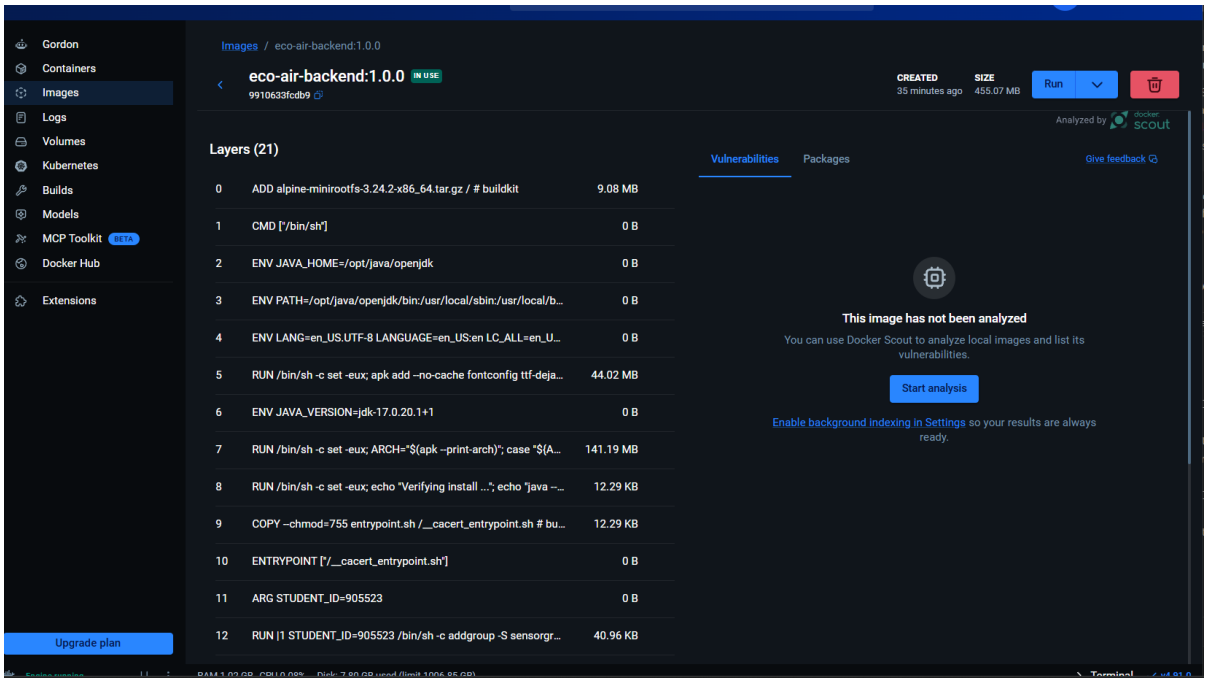
Datos de identidad de la imagen y del contenedor

Dato verificado	Resultado
Etiqueta de imagen	`eco-air-backend:1.0.0`
ID de imagen Docker	`sha256:9910633fcdb9e34c5f0d3333bdb1f5d1241fa453b461f5d412839f7d3c22aae4`
Digest de configuración reportado por BuildKit	`sha256:bf80f39a1bcc8c664bddbcdf592133e4f07368f5db0a6021999e80bb647115cd`
`OPERATOR_ID` dentro del contenedor	905523
ID de daemon Docker Desktop	`bae1e2b7-bdf6-48a8-8530-a671cb4b6c54`
Contenedor final	`eco_air_active`; ID `44ded8cbb97243a3920036952f390b7fdc9b95a56028bdf21c6072c85886351b`

Nota. El ID sha256 de la imagen y el digest de configuración de BuildKit son identificadores distintos, ambos reproducidos de sus respectivas salidas.

Figura 2

Capas de la imagen eco-air-backend:1.0.0 en Docker Desktop



Nota. Captura de pantalla de Docker Desktop del 20 de octubre de 2026. Se observan las capas de la imagen, incluidos `ENV JAVA_HOME`, `ARG STUDENT_ID=905523` y la creación del usuario del sistema.

Reto 2: auditoría del runtime

La Tabla 2 reúne los comandos ejecutados sobre el contenedor `eco_air_active` y el resultado observado en cada uno.

Tabla 2
Auditoría del runtime del contenedor

Comando	Resultado observado	Significado
`docker top eco_air_active -o pid,user,comm`	`875 10001 java`	El daemon muestra el UID numérico del proceso.
`docker exec eco_air_active id`	`uid=10001(sensor_user) gid=101(sensorgroup) groups=101(sensorgroup)`	Confirma el nombre y los identificadores del usuario y del grupo.
`docker exec eco_air_active touch /etc/crontab_fake`	`touch: cannot touch '/etc/crontab_fake': Permission denied` (código 1)	El usuario no tiene permiso para escribir en esa ubicación del sistema.
`docker exec eco_air_active which mvn`	Sin salida; código 1.	Maven no está instalado en la etapa runtime.
`docker exec eco_air_active sh -c 'find / -name "*.java" 2>/dev/null'`	Sin salida; código 1.	No se localizaron fuentes Java; el código 1 se conserva como resultado observado.

Nota. Los comandos se ejecutaron sobre el contenedor final `eco_air_active`.

La salida de `docker history` confirmó la presencia de la capa del JAR, la creación del usuario, las variables de entorno y el `ENTRYPOINT` en la imagen final. Las cadenas largas de algunas capas aparecen truncadas por el formato legible de `docker history`; la salida íntegra capturada está en `audit_session.log`.

La evidencia permite concluir que la compilación y el runtime están desacoplados: el proceso se ejecuta como usuario no privilegiado, la escritura de prueba fue denegada y no se encontraron ni Maven ni archivos `.java` en el runtime. Estas son observaciones concretas de la imagen construida, no una afirmación de que toda posible vulnerabilidad haya sido auditada.

Reto 3: prueba de OOM y calibración

Terminación por memoria a 48 MiB

```
docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0
# ID observado:
24aeb41f581e3a5f72bdbee483835d134598bb702b2011a50c4ed12bbd8b2eff

docker inspect eco_air_crash --format='Status: {{.State.Status}} |
ExitCode: {{.State.ExitCode}} | OOMKilled: {{.State.OOMKilled}}'

Status: exited | ExitCode: 137 | OOMKilled: true
```

A los 10 segundos el contenedor todavía aparecía como `running`; después de esperar otros 20 segundos, la inspección confirmó `exited`, código 137 y `OOMKilled=true`. Por tanto, el resultado no fue una salida inventada ni una inferencia basada solo en la memoria consumida: Docker informó explícitamente la terminación por OOM. La guía de Docker advierte que los procesos que agotan la memoria pueden ser terminados por el kernel y recomienda probar los requisitos antes de fijar límites de producción (Docker Inc., s. f.-b).

Calibración observada

La Tabla 3 presenta los límites de memoria ensayados, el resultado y los valores observados.

Tabla 3
Calibración del límite de memoria del contenedor

Límite	Resultado	Arranque y memoria observada
48 MiB	No llegó a completar el arranque; `OOMKilled=true`, código 137.	47.91 MiB / 48 MiB; 99.81 % a los 10 s; después del OOM, 0 B / 0 B.
112 MiB	`running` en la observación, pero no cumple < 15 s.	16.508 s; 106.6 MiB / 112 MiB (95.20 %).
128 MiB	`running`; cumple el umbral de arranque y salud.	9.294 s en la calibración; 9.824 s en la verificación final. Lecturas registradas: 122.3, 123.1, 121.1 y 122 MiB / 128 MiB.
144 MiB	`running`.	7.559 s; 126.6–126.9 MiB / 144 MiB.
160 MiB	`running`.	7.064 s; 143.3–143.5 MiB / 160 MiB.
192 MiB	`running`.	6.132 s; 175.3 MiB / 192 MiB.
256 MiB	`running`.	6.838 s; 178.2 MiB / 256 MiB.
384 MiB	`running`.	7.948 s; 177.6–178.7 MiB / 384 MiB.
512 MiB	`running`.	5.941 s; 179.6 MiB / 512 MiB.

Nota. Los tiempos proceden de la línea “Started EcoAirApplication in … seconds” de los logs. Las cifras de memoria son lecturas reales de `docker stats`; cuando se indican varios valores, corresponden a capturas distintas y no a un rango estimado.

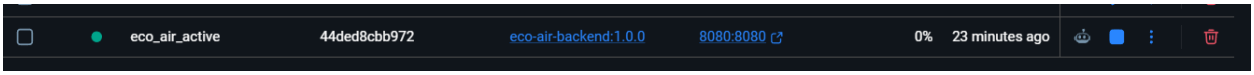
El límite elegido fue 128 MiB: es el menor valor probado que inició en menos de 15 segundos y permaneció en ejecución durante las observaciones posteriores. A 112 MiB el proceso seguía activo al momento de una inspección, pero tardó 16.508 segundos en alcanzar

`Started EcoAirApplication`, de modo que no cumplió el criterio temporal. La conclusión se limita a los valores ensayados y a la carga de esta ejecución; no afirma que 128 MiB sea un mínimo universal para cualquier carga o equipo.

Estado final y salida completa de docker stats

El contenedor final fue recreado desde la etiqueta de la imagen después de la última construcción. Quedó ejecutándose como `eco_air_active`, con 128 MiB de límite y publicación 8080:8080 (ver Figura 3). Los logs registraron “Started EcoAirApplication in 9.824 seconds (process running for 10.771)”; `OPERATOR_ID` fue 905523, y `/actuator/health` respondió `{"status":"UP"}`.

Figura 3
Contenedor eco_air_active en ejecución en Docker Desktop



Nota. Captura de pantalla de Docker Desktop. El contenedor `eco_air_active` usa la imagen `eco-air-backend:1.0.0` y publica el puerto 8080:8080.

La Tabla 4 presenta la salida completa de `docker stats`, capturada el 20 de octubre de 2026 a las 21:02:40 (UTC−05:00).

Tabla 4
Salida completa de docker stats para eco_air_active

Campo	Valor exacto	Campo	Valor exacto
`CONTAINER ID`	`44ded8cbb972`	`NAME`	`eco_air_active`
`CPU %`	0.63 %	`MEM USAGE / LIMIT`	122 MiB / 128 MiB
`MEM %`	95.33 %	`NET I/O`	2.99 kB / 1.77 kB
`BLOCK I/O`	144 MB / 187 MB	`PIDS`	31

Nota. La medición registra un uso alto: 95.33 % del límite. Aun así, en esa captura el contenedor estaba `running`, `OOMKilled=false`, y la comprobación de salud respondió UP. La memoria de una aplicación y la salida de `docker stats` pueden variar con el tiempo; la tabla representa esta lectura puntual, no una garantía de margen permanente.

Ejecución y comprobación para el usuario

Usar el contenedor Docker existente

Desde PowerShell, en cualquier directorio con el contexto Docker activo, si el contenedor ya existe pero está detenido, se inicia con `docker start`. No se debe crear un segundo contenedor con el mismo nombre ni pulsar Run en la página de la imagen si se desea reutilizar `eco_air_active`.

```
docker start eco_air_active
docker ps --filter "name=eco_air_active"
curl.exe http://localhost:8080/actuator/health
docker logs --tail 50 eco_air_active
```

La salida esperada de la comprobación de salud es `{"status":"UP"}`. Si `docker ps` ya muestra `eco_air_active` como `Up`, no es necesario volver a iniciarlo.

Ejecutar en modo de desarrollo local con Maven

Docker y Spring Boot local intentan escuchar en el puerto 8080; por ello deben ejecutarse de forma alternada, no simultánea. Para usar Maven desde la raíz del proyecto, primero se detiene el contenedor y luego se ejecuta el goal correcto del plugin de Spring Boot. El objetivo `run` forma parte del plugin de Maven; `mvn run`, sin el prefijo `spring-boot`, no es una fase válida del ciclo de vida de Maven (VMware, s. f.).

```
docker stop eco_air_active
mvn spring-boot:run
# Cuando termine la prueba local: Ctrl+C
# Para volver a Docker:
docker start eco_air_active
```

En la ejecución local compartida, Maven compiló y Spring comenzó a inicializar, pero falló porque el puerto 8080 seguía ocupado. La salida reportó “Web server failed to start. Port 8080 was already in use.” La ejecución local se identificó como Java 25; el contenedor del taller utiliza Java 17, por lo que no deben confundirse ambos entornos.

Guion breve para la exposición

Construí EcoAir Backend con un Dockerfile multietapa: Maven compila en la etapa `builder` y la imagen final conserva el JRE y el JAR, sin Maven ni fuentes Java. La aplicación corre como `sensor_user` con UID 10001 y recibe mi identificador 905523 en `OPERATOR_ID`. Comprobé la restricción de escritura en `/etc` y calibré la memoria del contenedor. Con 48 MiB, Docker reportó `OOMKilled=true` y código 137; 112 MiB no cumplió el inicio menor de 15 segundos; 128 MiB fue el menor límite probado que sí lo cumplió. La verificación final respondió UP, aunque las estadísticas mostraron 122 de 128 MiB, así que el margen de memoria es reducido. Todas las cifras se respaldan con el log y el informe de auditoría.

Conclusiones

- La construcción multietapa separó las herramientas de compilación del entorno de ejecución y permitió copiar solo el JAR al runtime.
- Las comprobaciones de UID, permisos, Maven y fuentes Java respaldaron el aislamiento básico esperado para la imagen final.
- El resultado a 48 MiB confirmó una terminación por OOM. La calibración mostró que satisfacer el tiempo de arranque y mantenerse activo requiere más memoria que ese límite; 128 MiB fue el mínimo entre los valores ensayados que cumplió el criterio temporal.
- La lectura final de 122 MiB sobre 128 MiB advierte que el umbral es funcional en esta prueba, pero deja poco margen y debe interpretarse como evidencia de laboratorio, no como recomendación de producción.

- La identidad de imagen, el estado, la salud, el entorno y las estadísticas se registraron en Docker Desktop local. Los contenedores ajenos especificados al inicio no se eliminaron ni se modificaron.

Referencias

Docker Inc. (s. f.-a). *Multi-stage builds*. Docker Docs. https://docs.docker.com/build/building/multi-stage/

Docker Inc. (s. f.-b). *Resource constraints*. Docker Docs. https://docs.docker.com/engine/containers/resource_constraints/

VMware. (s. f.). *Running your application with Maven*. Spring Boot Maven Plugin. https://docs.spring.io/spring-boot/maven-plugin/run.html

Anexo A

Archivos de evidencia

El repositorio conserva la transcripción `audit_session.log` con las ejecuciones, marcas de tiempo y salidas del terminal, junto con `FORENSIC_REPORT.md`, que resume la identidad de la imagen, la ejecución, el OOM y la calibración. La carpeta `taller-docker-semana13/` contiene el Dockerfile, `.dockerignore`, el log y el informe; los archivos `taller_docker_semana13.pdf` y `taller_docker_semana13.zip` son los entregables de la práctica. El estado de Git se verificó limpio y `main` quedó alineada con `origin/main` en el commit `ba2c94c`.